Engineering Take-Home: Webhook Delivery Service

Background

Most SaaS products eventually need to send webhooks — HTTP callbacks fired when something interesting happens, delivered reliably to subscriber-controlled URLs. Doing this well is harder than it looks. You'll build a small version of such a service.

What to build

A service that:

1. Lets clients **subscribe** to events by registering a target URL, an optional shared secret, and a filter on event types (e.g. `order.created`, `user.*`).
2. Accepts **events** via an ingest endpoint and fans them out to matching subscriptions.
3. **Delivers** events to subscriber URLs with retries on failure, using a configurable backoff and max-attempt policy.
4. **Persists** events and delivery attempts so nothing is lost across process restarts.
5. Exposes a **dashboard** (web UI) to list subscriptions, browse recent events, drill into delivery attempts for a given event, and manually retry a failed delivery.

Explicit non-goals

To keep scope honest — do **not** build:

- Real authentication (a single shared admin key is fine).
- Multi-tenancy or RBAC.
- A distributed system. Single process is expected.
- Production deployment infra. Local `docker compose up` or `npm start` is enough.

Constraints

- **Stack: your choice.** Any language, any framework, any database. SQLite is perfectly fine; pure in-memory is not (events must survive restart).
- **One repo**, runnable with a single documented command.
- **Time-boxed**: aim for 4-6 hours spread over a few days. If you go over, stop and write down what you'd do next — we'd rather see good judgment about scope than an exhausted candidate.

Deliverables

Submit a git repo (zip or link) containing:

1. **Source code.**
2. **`README.md`** — how to run, what works, what's incomplete, what you'd improve with more time.
3. **`DECISIONS.md`** — one page maximum. For each of the following, a short paragraph on what you chose, what alternatives you considered, and why: **storage**, **concurrency / worker model**, **retry policy**, **payload signing**, **dashboard scope**.
4. **`AI_LOG.md`** — five to ten meaningful entries documenting your significant AI interactions. For each: what you asked, what came back, and what you kept, modified, or rejected. Don't paste full transcripts; we want to see your judgment, not your prompt history. If you rejected nothing, we'll be skeptical.

What we're looking for

We're not grading on feature count. We're looking at:

- **Completeness over breadth.** A smaller system that actually works — retries genuinely retry, restart genuinely recovers, the dashboard genuinely reflects state — beats a wider system that's half-wired.
- **Design judgment.** The `DECISIONS.md` is where you show this. We want to see you understood the tradeoffs of what you picked, not that you picked the "right" answer.
- **Code quality.** Readable, sensibly organized, with enough tests to demonstrate the critical paths work. Not exhaustive coverage.
- **AI as a tool, not a crutch.** We expect you to use AI. We also expect you to understand every line you ship and be able to defend it. The `AI_LOG.md` and the walkthrough will surface this.

After submission: 30-min walkthrough

We'll schedule a short call where you'll:

- Walk us through your code (we'll pick the files).
- Discuss a couple of your design decisions.
- Extend the system live with a small change we'll describe on the call. AI is allowed during this.

The walkthrough is part of the evaluation. Code you can't navigate or extend doesn't count for much.

Hints (read once you've thought about it for a bit)

- Don't design microservices for this. One process, clear modules, clean boundaries.
- The genuinely interesting questions: What's your worker model — async loop, thread pool, separate process? What does "at least once" mean concretely in your implementation? What happens to a delivery in flight when the process crashes mid-request? How do you sign payloads so subscribers can verify them? How do you avoid retry storms when a subscriber is down?
- A subscriber URL that returns 200 means delivered. 4xx (except 408/429) usually means don't retry. 5xx and network errors mean retry with backoff. These rules are guidance, not gospel — make a call and explain it.
- The dashboard can be plain. A table view and a detail view is enough. We're not grading CSS.

Questions

If something is genuinely ambiguous, make a call, document it in `DECISIONS.md`, and move on. We'd rather see you exercise judgment than wait for clarification.

Good luck.